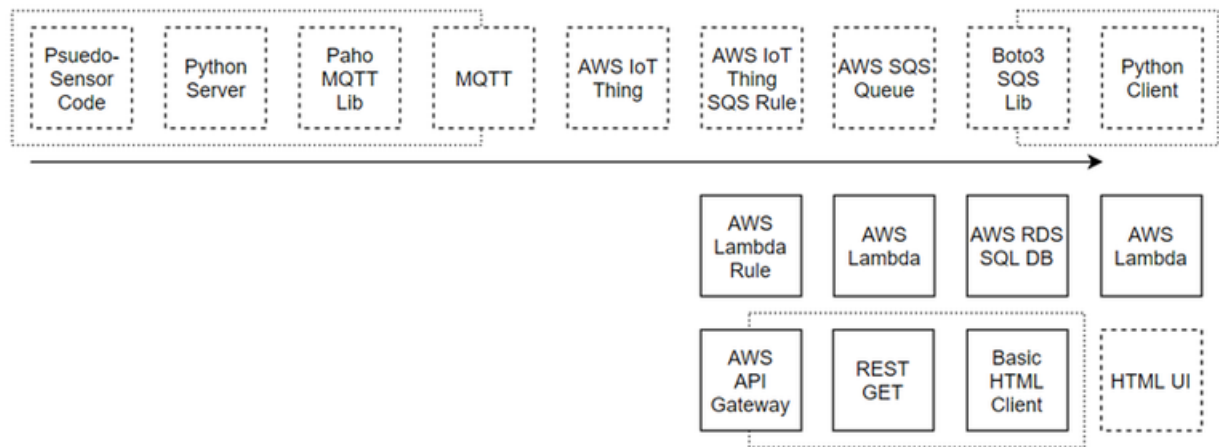




Project Requirements

The requirements for the second assignment are as follows:

- 1) Using the code from your previous Project 1, add a rule to the AWS IoT Thing to send incoming data to AWS Lambda (as well as the existing rule for sending data to the SQS Queue). You can still visit the AWS SQS Queue in the AWS console to see your messages arriving from your connected Python data server.
- 2) Using either Node.JS or Python, create a Lambda rule that routes the incoming messages to an AWS database of your choice.
- 3) Develop a Microservice that will provide a REST API to request data from the data stored in the AWS database. You can decide how you will request data (get the latest value, allow for a query, etc.). Your Microservice will use AWS API Gateway and another Lambda program, as well as the AWS database you chose, to combine to provide this functionality.
- 4) Create a simple client in Python to exercise the REST API you created in part 3, and demonstrate requesting and receiving data from the AWS database.

Optionally, you could use the HTML UI client developed in the Rapid Prototyping course as a way to display the data you are receiving from the REST API and Microservice, but that is not strictly required (although it would be fun to do)! If you do this, capture an image of your HTML UI displaying the received data as it comes in.

Implementation/Assumption Notes

Pseudo-Sensor → AWS IoT → SQS Pipeline

Pipeline architecture:

Pseudo-Sensor Code → Python Server → Paho MQTT Lib → MQTT → AWS IoT Thing
 → AWS IoT Thing SQS Rule → AWS SQS Queue → Boto3 SQS Lib → Python Client

Target device: Raspberry Pi 4B (aarch64, Raspberry Pi OS / Debian 13.

AWS account: 596633517506, region eu-central-1.

IoT Thing: RasPi4B-VZ-180726

■ **psuedoSensor.py** — the PseudoSensor class (unchanged from the original code), cycling through fixed humidity/temperature ranges with random jitter.

sensor_server.py — a long-running loop:

Builds a reading dict {"humidity": ..., "temperature": ..., "ts": <ISO8601 UTC>} every --interval seconds (default 5s).

■ **Paho MQTT Lib → MQTT → AWS IoT Thing**

Policy update

The existing device policy (RasPi4B-VZ-180726-Policy) only allowed the sample topics (sdk/test/*) and client IDs (basicPubSub, etc.) used by the AWS SDK sample. It was extended to also allow:

Publish/Receive on topic sensors/raspi4b/telemetry

Subscribe on topic filter sensors/raspi4b/telemetry

Connect as client raspi4b-sensor-server

Applied via:

```
aws iot create-policy-version \  
  --policy-name RasPi4B-VZ-180726-Policy \ ..
```

Publisher module

mqtt_publisher.py — MqttPublisher class:

Endpoint: a3dp4umq4qv6ul-ats.iot.eu-central-1.amazonaws.com:8883

Client ID: raspi4b-sensor-server

Topic: sensors/raspi4b/telemetry

TLS: mutual auth using root-CA.crt + RasPi4B-VZ-180726.cert.pem + RasPi4B-VZ-XXX.private.key

Publishes each reading as JSON with QoS 1, waiting for the publish to complete.

SQS queue

aws sqs create-queue --queue-name raspi4b-telemetry-queue

Queue ARN: arn:aws:sqs:eu-central-1:596633517506:raspi4b-telemetry-queue Queue URL:

<https://sqs.eu-central-1.amazonaws.com/596633517506/raspi4b-telemetry-queue>

■ IAM role for the Rules Engine

The Rules Engine needs explicit permission to write to the queue. Two parts:

Trust policy and **Permissions policy**

Boto3 SQS Lib → Python Client

sqs_client.py — a boto3-based consumer, run on the same Raspberry Pi:

Long-polls raspi4b-telemetry-queue (receive_message with MaxNumberOfMessages=10,

WaitTimeSeconds=20) — long polling avoids busy-waiting and empty-response cost compared to short polling.

AWS RDS (Relational Data Base)– PostgreSQL

Created within AWS IoT Console:

The screenshot displays the AWS RDS console for a PostgreSQL instance named 'database-coursera-iot'. The left sidebar shows navigation options like Dashboard, Databases, Performance insights, Snapshots, Exports in Amazon S3, Automated backups, Reserved instances, Proxies, Subnet groups, Parameter groups, Option groups, Custom engine versions, Zero-ETL integrations, Events, Event subscriptions, Recommendations, and Certificate update.

The main content area shows the instance details under the 'Summary' tab. Key information includes:

- DB identifier:** database-coursera-iot
- Status:** Available
- Role:** Instance
- Engine:** PostgreSQL
- Region & AZ:** eu-central-1b
- Current activity:** 0.00 sessions
- CPU:** 4.07%
- Class:** db.t4g.micro

Below the summary, there are tabs for Connectivity & security, Monitoring, Logs & events, Configuration, Zero-ETL integrations, Maintenance & backups, Data migrations, Tags, and Recommendations.

The 'Connectivity & security' tab is active, showing the 'Connect using' section with options for Code snippets (selected), CloudShell, and Endpoints. The 'Internet access gateway' is disabled. The 'IAM Authentication' is also disabled.

The 'Programming language' is set to 'PSQL (macOS)' and the 'Endpoint type' is 'Instance endpoint'.

The 'Connection steps' section provides the following commands:

```
1 curl -o global-bundle.pem https://truststore.pk1.rds.amazonaws.com/global/global-bundle.pem
2 export RDSHOST="database-coursera-iot.c5kcmqaau43.eu-central-1.rds.amazonaws.com"
3 psql "host=$RDSHOST port=5432 dbname=postgres user=postgres sslmode=verify-full sslrootcert=./global-bundle.pem"
```

The 'Connected compute resources (2)' section shows two Lambda functions connected to the instance:

Resource identifier	Resource type	Availability Zone	VPC security group	Compute resource security group	Connected proxy
ConnectPostgreSQL	Lambda function	Multiple AZs	rds-lambda-2	lambda-rds-2	-
GetTelemetry	Lambda function	Multiple AZs	rds-lambda-2	lambda-rds-3	-

The 'Security group rules (5)' section shows the default security group rule for the instance.

Two Lambda created for the required REST API :

1) **ConnectPostgreSQL** – Lambda function with logic to insert the received data from pseudoSensor.py to the table of PostgreSQL database “postgres”

2) **GetTelemetry** – Lambda Function with logic to query data from the AWS PostgreSQL and feed REST API with the telemetry data

Both Lambda functions were packed with required import packages and zipped. The resulted zip files were uploaded to the “Code source” tab of Lambda.

The necessary security groups were attached to both Lambdas:

The screenshot displays the AWS VPC console for a security group named 'sg-0f131df143994d36c'. The left sidebar shows navigation options like VPC dashboard, Virtual private cloud, Security, Internet gateways, Egress-only internet gateways, Carrier gateways, DHCP option sets, Elastic IPs, Managed prefix lists, NAT gateways, Peering connections, Route servers, and Security.

The main content area shows the details of the security group under the 'Details' tab. Key information includes:

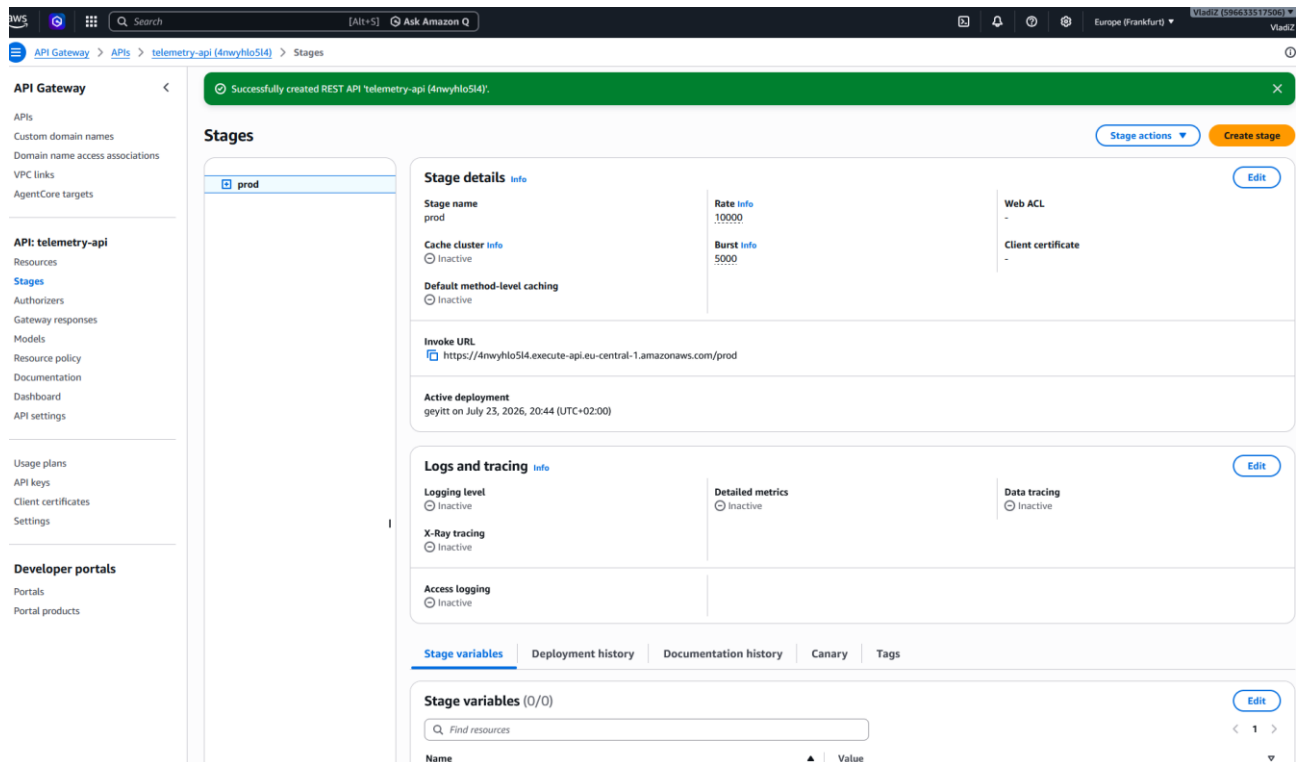
- Security group name:** rds-lambda-2
- Security group ID:** sg-0f131df143994d36c
- Description:** Security group attached to database-coursera-iot to allow a Lambda function with specific security groups attached to connect to the RDS database. Modification could lead to connection loss.
- Owner:** 596633517506
- Inbound rules count:** 2 Permission entries
- Outbound rules count:** 0 Permission entries

Below the details, there are tabs for Inbound rules, Outbound rules, Sharing, VPC associations, Related resources - new, and Tags.

The 'Inbound rules (2)' section shows the following rules:

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sgr-0d685231a561891ff	-	PostgreSQL	TCP	5432	sg-0f798b0e060a3623e...	Rule to allow connections from Lambda function with sg-0f798b0...
-	sgr-015b492c7e7482f88	-	PostgreSQL	TCP	5432	sg-03b27d779356e2beb...	Ingress rule for allowing lambda function to connect to rds db

REST API was created online within API Gateway:



HTML UI Client Code:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="utf-8">
```

```
<title>Telemetry</title>
```

```
<style>
```

```
body { font-family: system-ui, sans-serif; max-width: 60rem; margin: 2rem auto; padding: 0 1rem; }
```

```
h1 { font-size: 1.3rem; }
```

```
.controls { display: flex; gap: .75rem; align-items: end; margin-bottom: 1rem; flex-wrap: wrap; }
```

```
label { display: block; font-size: .8rem; color: #555; }
```

```
input, button { padding: .4rem .6rem; font: inherit; }
```

```
table { border-collapse: collapse; width: 100%; }
```

```
th, td { padding: .4rem .6rem; text-align: right; border-bottom: 1px solid #ddd; }
```

```
th:last-child, td:last-child { text-align: left; }
```

```
th { background: #f4f4f4; font-size: .8rem; }
```

```

#status { margin: .75rem 0; font-size: .85rem; color: #666; }

.err { color: #b00; }

</style>

</head>

<body>

<h1>Telemetry readings</h1>


<div class="controls">

  <div>

    <label for="url">API endpoint</label>

    <input id="url" size="56"

      value="https://4nwyhlo5l4.execute-api.eu-central-1.amazonaws.com/prod/telemetry">

  </div>

  <div>

    <label for="limit">Limit</label>

    <input id="limit" type="number" value="20" min="1" max="500" style="width:5rem">

  </div>

  <button id="load">Load</button>

  <label style="font-size:.85rem"><input type="checkbox" id="auto"> auto-refresh 10s</label>

</div>


<div id="status"></div>

<table>

  <thead><tr><th>ID</th><th>Humidity %</th><th>Temp °C</th><th>Reading
timestamp</th></tr></thead>

  <tbody id="rows"></tbody>

</table>


<script>

const $ = id => document.getElementById(id);

let timer = null;

```

```

async function load() {
  const url = new URL($('url').value);
  url.searchParams.set('limit', $('limit').value);
  $('status').textContent = 'Loading...';
  $('status').className = '';

  try {
    const res = await fetch(url);
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const data = await res.json();

    $('rows').innerHTML = data.readings.map(r => `
      <tr>
        <td>${r.id}</td>
        <td>${r.humidity.toFixed(2)}</td>
        <td>${r.temperature.toFixed(2)}</td>
        <td>${new Date(r.reading_ts).toLocaleString()}</td>
      </tr>`).join("");

    $('status').textContent =
      `${data.count} readings · updated ${new Date().toLocaleTimeString()}`;
  } catch (e) {
    $('status').textContent = `Failed: ${e.message}`;
    $('status').className = 'err';
  }
}

$('load').onclick = load;
$('auto').onchange = e => {
  clearInterval(timer);
  if (e.target.checked) timer = setInterval(load, 10000);
};

```

```
load();  
</script>  
</body>  
</html>
```

2) Python code:

pseudoSensor.py:

```
import random
```

```
class PseudoSensor:
```

```
    h_range = [0, 20, 20, 40, 40, 60, 60, 80, 80, 90, 70, 70, 50, 50, 30, 30, 10, 10]  
    t_range = [-20, -10, 0, 10, 30, 50, 70, 80, 90, 80, 60, 40, 20, 10, 0, -10]
```

```
    h_range_index = 0  
    t_range_index = 0
```

```
    humVal = 0  
    tempVal = 0
```

```
    def __init__(self):  
        self.humVal = self.h_range[self.h_range_index]  
        self.tempVal = self.t_range[self.t_range_index]
```

provides a new value for humidity and temperature based on the ranges defined above. The values are generated in a loop, so that they can be used to simulate a sensor reading over time.

```
    def generate_values(self):  
        self.humVal = self.h_range[self.h_range_index] + random.uniform(0, 10)  
        self.tempVal = self.t_range[self.t_range_index] + random.uniform(0, 10)
```

```
        self.h_range_index += 1  
        if self.h_range_index > len(self.h_range) - 1:  
            self.h_range_index = 0
```

```
        self.t_range_index += 1  
        if self.t_range_index > len(self.t_range) - 1:  
            self.t_range_index = 0
```

```
    return self.humVal, self.tempVal
```

mqttPublisher.py:

```
import json
import logging
import ssl

import paho.mqtt.client as mqtt

log = logging.getLogger("mqtt_publisher")

# MQTT configuration
ENDPOINT = "a3dp4umq4qv6ul-ats.iot.eu-central-1.amazonaws.com"
PORT = 8883
CLIENT_ID = "raspi4b-sensor-server"
TOPIC = "sensors/raspi4b/telemetry"

# TLS configuration
CA_CERT = "root-CA.crt"
CERT_FILE = "RasPi4B-VZ-180726.cert.pem"
KEY_FILE = "RasPi4B-VZ-180726.private.key"

# create a new MQTT client and configure TLS settings
def build_client():
    client = mqtt.Client(client_id=CLIENT_ID,
callback_api_version=mqtt.CallbackAPIVersion.VERSION2)
    client.tls_set(
        ca_certs=CA_CERT,
        certfile=CERT_FILE,
        keyfile=KEY_FILE,
        tls_version=ssl.PROTOCOL_TLSv1_2,
    )
    client.on_connect = _on_connect
    client.on_disconnect = _on_disconnect
    return client

# MQTT callback functions
def _on_connect(client, userdata, flags, reason_code, properties):
    log.info("Connected to %s (reason_code=%s)", ENDPOINT, reason_code)

def _on_disconnect(client, userdata, flags, reason_code, properties):
    log.warning("Disconnected from %s (reason_code=%s)", ENDPOINT, reason_code)

# MqttPublisher class: wraps the MQTT client and provides a simple interface for publishing
messages to a topic.
class MqttPublisher:
```

```

def __init__(self):
    self.client = build_client()
    self.client.connect(ENDPOINT, PORT)
    self.client.loop_start()

def publish(self, reading):
    payload = json.dumps(reading)
    result = self.client.publish(TOPIC, payload, qos=1)
    result.wait_for_publish(timeout=5)
    log.info("Published to %s: %s", TOPIC, payload)

def close(self):
    self.client.loop_stop()
    self.client.disconnect()

```

sqs_client.py:

```

import argparse
import logging
import time
from datetime import datetime, timezone

# import the PseudoSensor and MqttPublisher classes from their respective modules
from psuedoSensor import PseudoSensor
from mqtt_publisher import MqttPublisher

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger("sensor_server")

# build a reading from the sensor, rounding the values and adding a timestamp
def build_reading(sensor):
    humidity, temperature = sensor.generate_values()
    return {
        "humidity": round(humidity, 2),
        "temperature": round(temperature, 2),
        "ts": datetime.now(timezone.utc).isoformat(),
    }

# the main loop of the sensor server, which reads from the sensor and calls the on_reading
callback
def run(interval_seconds, on_reading):
    sensor = PseudoSensor()
    log.info("Sensor server started, publishing every %ss", interval_seconds)
    while True:
        reading = build_reading(sensor)

```

```

    on_reading(reading) # call the callback function with the reading
    time.sleep(interval_seconds)

def log_reading(reading):
    log.info("H=%.2f T=%.2f ts=%s", reading["humidity"], reading["temperature"], reading["ts"])

# the main entry point for the sensor server script
    if __name__ == "__main__":
        parser = argparse.ArgumentParser(description="Pseudo-sensor server loop")
        parser.add_argument("--interval", type=float, default=5.0, help="seconds between readings")
        parser.add_argument("--no-publish", action="store_true", help="log only, skip MQTT publish")
        args = parser.parse_args()

        if args.no_publish:
            on_reading = log_reading
        else:
            publisher = MqttPublisher()

            # define a callback function that logs the reading and publishes it to MQTT
            def on_reading(reading):
                log_reading(reading)
                publisher.publish(reading)

        try:
            run(args.interval, on_reading)
        except KeyboardInterrupt:
            log.info("Sensor server stopped")

```

Lambda ConnectPostgreeSQL

Python code of Lambda „ **ConnectPostgreeSQL** ” :

```

import json
import os
import psycopg2
import logging

logging.getLogger().setLevel(logging.INFO)
log = logging.getLogger()

_conn = None

def get_connection():

```

```

global _conn
if _conn is None or _conn.closed:
    log.info("Connecting to %s:%s", os.environ['DB_HOST'], os.environ['DB_PORT'])
    _conn = psycopg2.connect(
        host=os.environ['DB_HOST'],
        port=int(os.environ['DB_PORT']),
        dbname=os.environ['DB_NAME'],
        user=os.environ['DB_USER'],
        password=os.environ['DB_PASSWORD'],
        sslmode='require',
        connect_timeout=5,
    )
    log.info("Connected OK")
return _conn

def lambda_handler(event, context):
    log.info("Handler start")
    conn = get_connection()
    with conn.cursor() as cur:
        cur.execute(
            "INSERT INTO telemetry (humidity, temperature, reading_ts) VALUES (%s, %s, %s)",
            (event["humidity"], event["temperature"], event["ts"]),
        )
    conn.commit()
    log.info("Insert committed")
    return {"statusCode": 200, "body": json.dumps({"message": "Inserted"})}

```

Lambda Function GetTelemetry:

```

import json

import os
import logging
import psycopg2

logging.getLogger().setLevel(logging.INFO)
log = logging.getLogger()

_conn = None

def get_connection():
    global _conn
    if _conn is None or _conn.closed:

```

```

    _conn = psycopg2.connect(
        host=os.environ['DB_HOST'],
        port=int(os.environ['DB_PORT']),
        dbname=os.environ['DB_NAME'],
        user=os.environ['DB_USER'],
        password=os.environ['DB_PASSWORD'],
        sslmode='require',
        connect_timeout=5,
    )
    return _conn

def _response(status, body):
    return {
        "statusCode": status,
        "headers": {
            "Content-Type": "application/json",
            "Access-Control-Allow-Origin": "*",
        },
        "body": json.dumps(body, default=str),
    }

def _query(sql, args):
    global _conn
    try:
        conn = get_connection()
        with conn.cursor() as cur:
            cur.execute(sql, args)
            return cur.fetchall()
    except psycopg2.OperationalError:
        _conn = None          # stale socket after container thaw
        conn = get_connection()
        with conn.cursor() as cur:
            cur.execute(sql, args)
            return cur.fetchall()

def lambda_handler(event, context):
    params = event.get("queryStringParameters") or {}

    try:
        limit = int(params.get("limit", 20))
    except ValueError:
        return _response(400, {"error": "limit must be an integer"})
    limit = max(1, min(limit, 500))

    since = params.get("since")

```

```

sql = "SELECT id, humidity, temperature, reading_ts, received_at FROM telemetry"
args = []
if since:
    sql += " WHERE reading_ts >= %s"
    args.append(since)
sql += " ORDER BY reading_ts DESC LIMIT %s"
args.append(limit)

try:
    rows = _query(sql, args)
except psycopg2.Error as e:
    log.error("DB error: %s", e)
    return _response(500, {"error": "database error"})

return _response(200, {
    "count": len(rows),
    "readings": [
        {
            "id": r[0],
            "humidity": float(r[1]),
            "temperature": float(r[2]),
            "reading_ts": r[3].isoformat(),
            "received_at": r[4].isoformat(),
        }
        for r in rows
    ],
})

```

3) Logs and Screenshots:

Publisher Log:

```

2026-07-20 10:11:55,456 INFO Sensor server started, publishing every 2.0s
2026-07-20 10:11:55,457 INFO H=0.38 T=-18.17 ts=2026-07-20T08:11:55.456929+00:00
2026-07-20 10:11:55,526 INFO Connected to a3dp4umq4qv6ul-ats.iot.eu-central-1.amazonaws.com
(reason_code=Success)
2026-07-20 10:11:55,542 INFO Published to sensors/raspi4b/telemetry: {"humidity": 0.38,
"temperature": -18.17, "ts": "2026-07-20T08:11:55.456929+00:00"}
2026-07-20 10:11:57,543 INFO H=20.21 T=-7.69 ts=2026-07-20T08:11:57.543068+00:00
2026-07-20 10:11:57,578 INFO Published to sensors/raspi4b/telemetry: {"humidity": 20.21,
"temperature": -7.69, "ts": "2026-07-20T08:11:57.543068+00:00"}
2026-07-20 10:11:59,578 INFO H=23.49 T=3.27 ts=2026-07-20T08:11:59.578515+00:00
2026-07-20 10:11:59,610 INFO Published to sensors/raspi4b/telemetry: {"humidity": 23.49,
"temperature": 3.27, "ts": "2026-07-20T08:11:59.578515+00:00"}
2026-07-20 10:12:01,611 INFO H=45.25 T=11.42 ts=2026-07-20T08:12:01.611045+00:00

```

2026-07-20 10:12:01,639 INFO Published to sensors/raspi4b/telemetry: {"humidity": 45.25, "temperature": 11.42, "ts": "2026-07-20T08:12:01.611045+00:00"}

2026-07-20 10:12:03,639 INFO H=41.13 T=33.33 ts=2026-07-20T08:12:03.639423+00:00

2026-07-20 10:12:03,677 INFO Published to sensors/raspi4b/telemetry: {"humidity": 41.13, "temperature": 33.33, "ts": "2026-07-20T08:12:03.639423+00:00"}

2026-07-20 10:12:05,678 INFO H=62.48 T=52.78 ts=2026-07-20T08:12:05.678306+00:00

2026-07-20 10:12:05,712 INFO Published to sensors/raspi4b/telemetry: {"humidity": 62.48, "temperature": 52.78, "ts": "2026-07-20T08:12:05.678306+00:00"}

SQS client Log:

2026-07-20 10:13:28,339 INFO Found credentials in shared credentials file: ~/.aws/credentials

2026-07-20 10:13:28,482 INFO Polling <https://sqs.eu-central-1.amazonaws.com/596633517506/raspi4b-telemetry-queue>

2026-07-20 10:13:28,628 INFO H=45.25 T=11.42 ts=2026-07-20T08:12:01.611045+00:00

2026-07-20 10:13:28,710 INFO H=23.49 T=3.27 ts=2026-07-20T08:11:59.578515+00:00

2026-07-20 10:13:28,731 INFO H=62.48 T=52.78 ts=2026-07-20T08:12:05.678306+00:00

2026-07-20 10:13:28,818 INFO H=41.13 T=33.33 ts=2026-07-20T08:12:03.639423+00:00

2026-07-20 10:13:28,925 INFO H=0.38 T=-18.17 ts=2026-07-20T08:11:55.456929+00:00

2026-07-20 10:13:29,018 INFO H=20.21 T=-7.69 ts=2026-07-20T08:11:57.543068+00:00

1) Assignment 1 Client and Server SQS

```
EXPLORER
CONNECT_DEVICE_PACKAGE [SSH: 192.168.178.53]
  > __pycache__
  > aws-iot-device-sdk-python-v2
  > iam
  > vemv
  > mqtt_publisher.py
  > pseudoSensor.py
  > publisher.log
  > Python-ENV.md
  > Rasp4B-VZ-180726-Policy
  > Rasp4B-VZ-180726.cert.pem
  > Rasp4B-VZ-180726.private.key
  > Rasp4B-VZ-180726.publickey
  > requirements.txt
  > root-CA.crt
  > sensor_server.py
  > Sensor-AWS-PahoMQTT-SQS.md
  > sq_client.py
  > sqs.log
  > start.sh

Python-ENV.md
sqs.log
sensor_server.py
mqtt_publisher.py
sq_client.py X
pseudoSensor.py

15 def process_message(body):
23
24 # Function to run the SQS client, which continuously polls the specified SQS queue for messages, processes them, and deletes them from the queue after processing.
25 def run(queue_url):
26     sqs = boto3.client("sqs", region_name=REGION) # Create an SQS client using Boto3
27     log.info("Polling %s", queue_url)
28     while True:
29         # Poll for messages from the SQS queue with long polling (WaitTimeSeconds=20)
30         response = sqs.receive_message(
31             QueueUrl=queue_url,
32             MaxNumberOfMessages=10,
33             WaitTimeSeconds=20,
34         )
35         # Get the list of messages from the response; if there are no messages, continue polling
36         messages = response.get("Messages", [])
37         if not messages:
38             continue
39         # Process each message in the list of messages, and delete it from the queue after processing
40         for message in messages:
41             process_message(message["Body"])
42             sqs.delete_message(
43                 QueueUrl=queue_url,
44                 ReceiptHandle=message["ReceiptHandle"],
45             )
46
47 # Entry point of the script; it sets up argument parsing for the queue URL and starts the SQS client.
48 if __name__ == "__main__":
49     parser = argparse.ArgumentParser(description="Boto3 SQS telemetry consumer")
50     parser.add_argument("--queue-url", default=QUEUE_URL)
51     args = parser.parse_args()
52
53     try:
54         run(args.queue_url)
55     except KeyboardInterrupt:
56         log.info("SQS client stopped")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR

vagrant@raspberrypi:~/aws-iot/connect-device-package$ timeout 12 ./venv/bin/python3 sensor_server.py --interval 2
2026-07-20 10:23:13,412 INFO Sensor server started, publishing every 2.0s
2026-07-20 10:23:13,412 INFO H=1.15 T=-15.12 ts=2026-07-20T08:23:13.412599+00:00
2026-07-20 10:23:13,467 INFO Connected to aldp4mq9sua1-ats.iot.eu-central-1.amazonaws.com (reason code=Success)
2026-07-20 10:23:15,500 INFO H=29.96 T=-3.72 ts=2026-07-20T08:23:15.500148+00:00
2026-07-20 10:23:15,531 INFO H=29.96 T=-3.72 ts=2026-07-20T08:23:15.500148+00:00
2026-07-20 10:23:17,533 INFO H=22.88 T=4.91 ts=2026-07-20T08:23:17.533530+00:00
2026-07-20 10:23:17,564 INFO H=22.88 T=4.91 ts=2026-07-20T08:23:17.533530+00:00
2026-07-20 10:23:19,564 INFO H=42.49 T=13.22 ts=2026-07-20T08:23:19.564795+00:00
2026-07-20 10:23:19,602 INFO H=42.49 T=13.22 ts=2026-07-20T08:23:19.564795+00:00
2026-07-20 10:23:21,602 INFO H=44.82 T=35.81 ts=2026-07-20T08:23:21.602851+00:00
2026-07-20 10:23:21,634 INFO H=44.82 T=35.81 ts=2026-07-20T08:23:21.602851+00:00
2026-07-20 10:23:23,634 INFO H=63.33 T=51.85 ts=2026-07-20T08:23:23.634745+00:00
2026-07-20 10:23:23,663 INFO H=63.33 T=51.85 ts=2026-07-20T08:23:23.634745+00:00
vagrant@raspberrypi:~/aws-iot/connect-device-package$ timeout 20 ./venv/bin/python3 sq_client.py
2026-07-20 10:24:01,525 INFO Found credentials in shared credentials file: ~/.aws/credentials
2026-07-20 10:24:01,616 INFO Polling https://sqs.eu-central-1.amazonaws.com/596633517506/rasp4b-telemetry-queue
2026-07-20 10:24:01,616 INFO H=29.96 T=-3.72 ts=2026-07-20T08:23:15.500148+00:00
2026-07-20 10:24:01,637 INFO H=63.33 T=51.85 ts=2026-07-20T08:23:23.634745+00:00
2026-07-20 10:24:01,930 INFO H=1.15 T=-15.12 ts=2026-07-20T08:23:13.412599+00:00
2026-07-20 10:24:01,957 INFO H=44.82 T=35.81 ts=2026-07-20T08:23:21.602851+00:00
2026-07-20 10:24:02,021 INFO H=42.49 T=13.22 ts=2026-07-20T08:23:19.564795+00:00
2026-07-20 10:24:02,169 INFO H=22.88 T=4.91 ts=2026-07-20T08:23:17.533530+00:00
vagrant@raspberrypi:~/aws-iot/connect-device-package$ ]
```

2) HTML UI and PostgresSQL query to telemetry db.

The screenshot displays a VNC session with a web browser and a terminal window. The browser shows a "Telemetry readings" page with a table of sensor data. The terminal window shows the execution of a Python script and a PostgreSQL query.

Telemetry readings

ID	Humidity %	Temp °C	Reading timestamp
11	83.05	99.43	7/23/2026, 6:53:22 PM
10	84.29	89.20	7/23/2026, 6:53:17 PM
9	62.34	77.82	7/23/2026, 6:53:12 PM
8	62.58	53.29	7/23/2026, 6:53:07 PM
7	43.26	32.44	7/23/2026, 6:53:02 PM
6	42.44	16.64	7/23/2026, 6:52:57 PM
5	27.06	3.83	7/23/2026, 6:52:52 PM
4	22.56	-9.44	7/23/2026, 6:52:47 PM
3	2.45	-12.38	7/23/2026, 6:52:41 PM
1	55.32	18.44	7/23/2026, 4:32:01 PM
2	55.32	18.44	7/23/2026, 4:32:01 PM

Python Script (sensor_server.py):

```
1 import json
2 import logging
3 import ssl
4
5 import paho.mqtt.client as mqtt
6
7 log = logging.getLogger("mqtt_publisher")
8
9 # MQTT configuration
10 ENDPOINT = "altpubmqtt-ats.iot.eu-central-1.amazonaws.com"
11 PORT = 8883
12 CLIENT_ID = "raspi4b-sensor-server"
13 TOPIC = "sensors/raspi4b/telemetry"
14
15 # TLS configuration
16 CA_CERT = "cert/root-CA.crt"
17 CERT_FILE = "cert/raspi4b-AC-188768.cert.pem"
18 KEY_FILE = "cert/raspi4b-AC-188768.private.key"
19
20 # Create a new MQTT client and configure TLS settings
21 def build_client():
22     client = mqtt.Client(client_id=CLIENT_ID, callback_api_version=mqtt.CallbackAPIVersion.VERSION2)
23     client.tls_set(ca_certs=CA_CERT, certfile=CERT_FILE, keyfile=KEY_FILE, tls_version=ssl.PROTOCOL_TLS)
```

Terminal Output:

```
2026-07-23 18:53:22.500 INFO Published to sensors/raspi4b/telemetry: {"humidity": 83.05, "temperature": 99.43, "ts": "2026-07-23T18:53:22.500Z"}
2026-07-23 18:53:22.500 INFO Published to sensors/raspi4b/telemetry: {"humidity": 84.29, "temperature": 89.20, "ts": "2026-07-23T18:53:22.500Z"}
2026-07-23 18:53:22.500 INFO Published to sensors/raspi4b/telemetry: {"humidity": 62.34, "temperature": 77.82, "ts": "2026-07-23T18:53:22.500Z"}
2026-07-23 18:53:22.500 INFO Published to sensors/raspi4b/telemetry: {"humidity": 62.58, "temperature": 53.29, "ts": "2026-07-23T18:53:22.500Z"}
2026-07-23 18:53:22.500 INFO Published to sensors/raspi4b/telemetry: {"humidity": 43.26, "temperature": 32.44, "ts": "2026-07-23T18:53:22.500Z"}
2026-07-23 18:53:22.500 INFO Published to sensors/raspi4b/telemetry: {"humidity": 42.44, "temperature": 16.64, "ts": "2026-07-23T18:53:22.500Z"}
2026-07-23 18:53:22.500 INFO Published to sensors/raspi4b/telemetry: {"humidity": 27.06, "temperature": 3.83, "ts": "2026-07-23T18:53:22.500Z"}
2026-07-23 18:53:22.500 INFO Published to sensors/raspi4b/telemetry: {"humidity": 22.56, "temperature": -9.44, "ts": "2026-07-23T18:53:22.500Z"}
2026-07-23 18:53:22.500 INFO Published to sensors/raspi4b/telemetry: {"humidity": 2.45, "temperature": -12.38, "ts": "2026-07-23T18:53:22.500Z"}
2026-07-23 18:53:22.500 INFO Published to sensors/raspi4b/telemetry: {"humidity": 55.32, "temperature": 18.44, "ts": "2026-07-23T18:53:22.500Z"}
2026-07-23 18:53:22.500 INFO Published to sensors/raspi4b/telemetry: {"humidity": 55.32, "temperature": 18.44, "ts": "2026-07-23T18:53:22.500Z"}
```

PostgreSQL Query:

```
postgres=# SELECT * FROM telemetry ORDER BY id DESC LIMIT 10;
 id | humidity | temperature | reading_ts | received_at
----+-----+-----+-----+-----
11 | 83.05 | 99.43 | 2026-07-23 18:53:22.500Z | 2026-07-23 18:53:22.640Z
10 | 84.29 | 89.20 | 2026-07-23 18:53:22.500Z | 2026-07-23 18:53:22.640Z
9 | 62.34 | 77.82 | 2026-07-23 18:53:22.500Z | 2026-07-23 18:53:22.640Z
8 | 62.58 | 53.29 | 2026-07-23 18:53:22.500Z | 2026-07-23 18:53:22.640Z
7 | 43.26 | 32.44 | 2026-07-23 18:53:22.500Z | 2026-07-23 18:53:22.640Z
6 | 42.44 | 16.64 | 2026-07-23 18:53:22.500Z | 2026-07-23 18:53:22.640Z
5 | 27.06 | 3.83 | 2026-07-23 18:53:22.500Z | 2026-07-23 18:53:22.640Z
4 | 22.56 | -9.44 | 2026-07-23 18:53:22.500Z | 2026-07-23 18:53:22.640Z
3 | 2.45 | -12.38 | 2026-07-23 18:53:22.500Z | 2026-07-23 18:53:22.640Z
2 | 55.32 | 18.44 | 2026-07-23 18:53:22.500Z | 2026-07-23 18:53:22.640Z
```